

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	P POORNA CHANDU
Name	192412199

COURSE OUTCOME COVERED

***CO2:** Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:50

Code of Conduct:

I P POORNA CHANDU(192412199) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation

- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	Marks 10 10 12.5 10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	

Decision Making	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	
-----------------	--	--	---	---	--

Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Criteria Max Marks

Clarity of

Response 7.5 Total 50

Answer 1: AI Drone Delivery in a Flood-Affected Region

i. Behavior of Greedy Best-First Search (GBFS)

Greedy Best-First Search expands the node whose heuristic value $h(n)$ — the straight-line distance to the medicine drop-off point — is smallest, ignoring the actual path cost $g(n)$ already incurred. In this scenario the drone would always fly toward the node that looks geometrically closest to the target, without weighing how costly or risky that segment currently is due to weather or terrain.

Strengths: Very fast, since it explores only the most 'promising' direction; low memory and computation, which suits the urgency requirement of quick decisions.

Weaknesses under uncertainty: It is not optimal and not complete under partial information. If a road that looks closest on the straight-line heuristic suddenly floods or gets blocked, GBFS has no built-in mechanism to reconsider cost — it can lead the drone into a dead end, force costly backtracking, or select a route that is geometrically short but practically dangerous (submerged terrain, high wind zones). Because it never looks at accumulated cost, it can also loop or oscillate between nodes that alternately look attractive when real-time updates change the map.

ii. Behavior of A* Search

A* expands nodes using $f(n) = g(n) + h(n)$, combining the actual cost incurred so far (g) with

the estimated remaining cost (h , straight-line distance). This lets it balance 'how far I have already travelled/risked' against 'how far I still have to go'.

Balancing cost and heuristic: A^* will not blindly chase the node nearest the destination; it also accounts for terrain difficulty and weather-inflated travel cost already spent, so a slightly longer but safer/cheaper path can be preferred over a short but currently expensive one.

Behaviour in this scenario: With an admissible and consistent heuristic (straight-line distance never overestimates true cost), A^* guarantees the shortest-cost, safest route is found while still pruning away clearly poor options — making it both optimal and reasonably efficient for the flood-relief problem, provided the heuristic and cost data are updated as new information arrives.

iii. Impact of Heuristic Accuracy

- Greedy Best-First Search: heuristic accuracy is everything — since cost is ignored entirely, an inaccurate straight-line estimate (which does not account for flooded roads or terrain) can send the drone down a badly suboptimal or unsafe path with no correction mechanism.
- A^* : a more accurate (but still admissible) heuristic makes A^* faster because fewer nodes are expanded, while still preserving optimality. An inaccurate but admissible heuristic makes A^* slower (more nodes explored) but the solution stays optimal. An inadmissible heuristic (overestimating cost) can make A^* miss the optimal route, exactly like GBFS.
- In short: accuracy improves GBFS's outcome quality directly (with no safety net), whereas for A^* it mainly affects search efficiency, not correctness, as long as admissibility holds.

iv. Effect of Dynamic Changes (Blocked Paths)

- Both algorithms operate on a static snapshot of the graph at the moment they search; when a road becomes newly blocked, previously computed paths can become invalid mid-flight.
- GBFS reacts poorly — since it never maintains a true cost record, replanning after a block often means restarting the greedy choice from scratch with no memory of a globally better route.
- A^* handles it better because it can be re-run incrementally (or via variants such as D^* / Lifelong Planning A^*) using the already-computed g -values, updating only the affected part of the search space rather than recomputing everything.
- Frequent dynamic changes increase computational overhead for both, but A^* 's cost tracking makes its replanned routes far more reliable and safer under uncertainty.

v. Recommended Algorithm

A^* Search (or a dynamic-replanning variant of A^* such as D^*) is the most suitable choice. It offers optimality guarantees together with a manageable computational cost, correctly balances the urgency (informed search converges faster than uninformed search) against

safety (it does not ignore the real, weather-affected travel cost the way Greedy Best-First Search does). Given that lives and emergency supplies are at stake, the small extra computation A* requires over GBFS is a reasonable trade-off for a route that is both quick to compute and far less likely to strand the drone on a bad path when real-time blocked-path updates arrive.

Answer 2: Smart City Traffic Signal Optimization

Formulation as an Optimization Problem

State: A configuration of signal timings (green/red durations and phase offsets) across all intersections at a given moment.

Objective function: A weighted combination to minimize: total vehicle waiting time + fuel consumption + congestion level, e.g. $f(\text{state}) = w_1 \cdot \text{WaitTime} + w_2 \cdot \text{Fuel} + w_3 \cdot \text{Congestion}$.

Search space: Every possible combination of timing values for every intersection — exponential in the number of intersections and phases, since each signal has a continuous/near-continuous range of durations.

Why traditional (uninformed/informed path-search) search is inefficient here: classic tree/graph search algorithms like BFS, DFS, or even A* are designed to find a path to a goal state through a sequence of discrete actions. This problem instead needs to find the best point in a huge, continuous, constantly shifting search space — there is no single 'goal node' to reach, and the state space is far too large to enumerate or search exhaustively in real time. This calls for local/optimization search methods that iteratively improve a candidate solution rather than search for a path.

i. Hill Climbing in this Scenario

Hill Climbing starts with an initial signal-timing configuration and repeatedly moves to a neighboring configuration (e.g., adjust one intersection's green time slightly) that improves the objective function, stopping when no neighbor improves it further.

Limitations: It only accepts improving moves, so it easily gets trapped in a state that is locally best but globally poor; it has no mechanism to escape once stuck, and with hundreds of intersections changing traffic patterns dynamically, a 'locally optimal' timing plan can quickly become outdated or clearly suboptimal city-wide.

ii. Local Maxima, Plateaus, and Ridges — Real-World Interpretation

- **Local maxima:** a signal-timing plan where every small adjustment makes waiting time worse, even though a very different plan elsewhere (e.g., re-timing an entire corridor together) would perform far better — the system settles for a 'good enough' but not truly optimal city configuration.
- **Plateaus:** regions where many neighboring timing adjustments produce almost identical waiting times (e.g., shifting a green phase by 1–2 seconds barely changes anything), causing Hill Climbing to wander without clear progress.

- Ridges: sequences of moves where no single-step adjustment improves things, but a coordinated multi-signal change (adjusting several intersections together, like a 'green wave') would — Hill Climbing, which only changes one variable at a time, cannot see this path forward.

iii. Simulated Annealing / Genetic Algorithms

Simulated Annealing: Occasionally accepts a worse move with a probability that decreases over time (governed by a 'temperature' schedule). This lets it escape local maxima and cross plateaus early in the search, while gradually settling into a strong solution as the temperature cools — well suited to a search space full of local optima like this one.

Genetic Algorithms: Maintain a population of candidate timing plans, combine (crossover) and mutate them across generations, and keep the fittest. Because many candidate solutions are explored in parallel, GAs are less likely to get stuck at any single local maximum and can discover coordinated, city-wide timing patterns that Hill Climbing's single-state, single-step search would miss.

iv. Recommended Approach

A Genetic Algorithm (optionally hybridized with local search / Simulated Annealing for fine tuning) is the best fit for scalability and adaptability: its population-based, parallel exploration scales naturally to hundreds of intersections, adapts well to continuously changing traffic patterns by periodically re-evolving the population with fresh traffic data, and is far less prone to getting stuck than plain Hill Climbing. Simulated Annealing is a strong lighter-weight alternative when computation time is more limited, since it needs to track only a single evolving solution rather than a full population.

Answer 3: Autonomous Mars Rover

i. Why an Online Search Agent is Needed

Offline search assumes the agent has complete knowledge of the environment before acting and computes an entire solution path in advance. The rover, however, does not have a full map of Mars terrain, cannot predict hidden hazards (craters, loose rocks) in advance, and has limited sensing range. It must interleave acting and sensing — take an action, observe the actual outcome, and use that new information to decide the next action. This 'act, observe, replan' loop is precisely what an online search agent provides, whereas an offline planner would produce a plan that could become invalid the moment reality diverges from its assumed map.

ii. Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover only knows about terrain within current sensor range, so it may commit to a direction that later reveals an impassable hazard, forcing costly backtracking.
- Dynamic/unknown terrain: obstacles are discovered only as the rover moves, so any

previously 'known-good' path can turn out to be blocked, and the rover must be able to revise its plan without restarting from scratch.

- Risk of irreversible actions: unlike simulated search, a wrong move (e.g., driving into a crater) can permanently damage the rover, so exploration strategies must be conservative and safety-aware, not just cost-minimizing.
- Limited communication with Earth (light-delay) also means the rover mostly cannot rely on real-time human correction — it must make autonomous local decisions.

iii. Building and Updating the Internal Model

The rover maintains an incrementally built map (often an occupancy grid or graph of visited/observed cells) that starts mostly unknown. As sensors detect terrain features, the model is updated with newly discovered obstacles, safe zones, and traversability costs. Techniques such as SLAM (Simultaneous Localization and Mapping) let the rover simultaneously track its own position and refine the terrain map from sensor readings, so each new observation both localizes the rover and expands what is known about the surroundings, which future planning steps then use.

iv. Online Search vs Uninformed/Informed Search

- Uninformed search (BFS, DFS, uniform-cost): assumes the full graph is known ahead of time and searches blindly without any estimate of goal distance — infeasible here since the map is not known upfront.
- Informed search (A^* , GBFS): improves efficiency using a heuristic, but classically still assumes the environment/graph is fully known before the search begins — a single offline computation of the whole path.
- Online search: does not require the map in advance; the agent explores physically, using local information plus a heuristic (when available, e.g., estimated distance to target coordinates) to decide the next move, and updates its knowledge after every action — essentially merging exploration and search into one continuous process, which matches the rover's real operating conditions.

v. Justification of the Most Suitable Approach

An online, informed search strategy — e.g., an online variant of A^* (such as LRTA*, Learning Real-Time A^*) — is most suitable. It combines the safety benefit of only committing to actions based on what is actually known/sensed (avoiding irreversible mistakes from an over-confident offline plan), the adaptability to revise the plan the instant new hazards are observed, and the efficiency of an informed heuristic to still bias exploration toward the target rather than wandering randomly like uninformed search. This balances the rover's core constraints of uncertainty, limited sensing, and mission safety.

Answer 4: University Exam Timetable Generation (CSP)

CSP Formulation

Variables: One variable per exam/course to be scheduled, e.g. $V = \{\text{Exam1}, \text{Exam2}, \dots, \text{ExamN}\}$, each representing 'when and where this exam is held'.

Domains: For each exam variable, the domain is the set of possible (time-slot, exam-hall) pairs available during the exam period, e.g. $\text{Domain}(\text{Exam}_i) = \{(\text{slot}, \text{hall}) : \text{slot} \in \text{available time slots}, \text{hall} \in \text{available halls with sufficient capacity}\}$.

Constraint Details

- No-overlap constraint (student clash): for any two exams sharing at least one common student, their assigned time slots must differ — a binary constraint between those exam variables.
- Hall-capacity constraint: for each time slot, the sum of students across exams scheduled in the same hall/slot must not exceed hall capacity; two exams cannot be assigned the same (slot, hall) pair unless combined enrollment fits.
- Precedence constraint: for subject pairs where one must precede another (e.g., prerequisite course exams), $\text{Slot}(\text{Exam}_A) < \text{Slot}(\text{Exam}_B)$.
- Faculty-availability constraint: an exam cannot be scheduled in a slot where its invigilating/setting faculty is unavailable.
- Special-accommodation constraint: students needing extra time or a separate room impose additional hall/slot restrictions on the exams they are enrolled in.

ii. How Backtracking Search Works Here

Backtracking search assigns exams to (slot, hall) pairs one variable at a time, checking after every assignment whether all constraints among the currently assigned variables are still satisfied. If an assignment violates a constraint (e.g., creates a student clash), the algorithm backtracks — undoes the most recent assignment(s) and tries a different value from the domain. This depth-first, one-variable-at-a-time approach systematically explores the space of possible timetables while pruning any branch that becomes inconsistent, avoiding the need to check every full combination exhaustively.

iii. Constraint Propagation (e.g., Forward Checking)

Forward checking looks ahead after each assignment: whenever an exam is scheduled into a slot/hall, it immediately removes now-invalid (slot, hall) values from the domains of all as yet-unassigned exams that are constrained with it (e.g., exams sharing students, or exams needing the same hall at that slot). This shrinks future search branches early, so the algorithm detects a dead end (an unassigned exam left with an empty domain) much sooner than plain backtracking would, which only discovers the conflict after actually trying the bad assignment. Combined with heuristics like Minimum Remaining Values (choose the most constrained exam next) and Least Constraining Value, this substantially reduces the search effort for large timetabling problems.

iv. Real-World Challenges and Best Strategy for Scalability

- As the number of courses/students grows, the constraint graph becomes dense (many exams share students), making pure backtracking too slow for large universities. • Combining backtracking with forward checking (or full arc-consistency, AC-3) plus good variable/value ordering heuristics (MRV, degree heuristic, LCV) is essential to keep the search tractable at scale.
- For very large instances, a hybrid approach — CSP formulation solved with local search/metaheuristics (e.g., min-conflicts, simulated annealing) after an initial greedy/backtracking assignment — scales better than exhaustive backtracking alone, because it can quickly repair a mostly-good timetable rather than searching from scratch.
- Overall, the best strategy for scalability is: CSP formulation + constraint propagation (forward checking/AC-3) + smart ordering heuristics, falling back to local repair search (min-conflicts) for very large problem sizes.

Answer 5: Strategic Game AI — Minimax and Alpha-Beta Pruning

i. How Minimax Models the Problem

Minimax models the game as a tree of alternating turns: the AI (maximizing player) tries to choose moves that maximize its evaluated outcome, while the opponent (minimizing player) is assumed to choose moves that minimize the AI's outcome (i.e., plays optimally against it). The algorithm recursively evaluates the tree from the leaves (terminal or depth-limited states, scored by an evaluation function) back up to the root: at MAX nodes it takes the maximum of children's values, at MIN nodes the minimum, and the AI ultimately selects the root move leading to the branch with the best guaranteed value assuming worst-case opponent play.

ii. Computational Challenges of Large Game Trees

- Exponential growth: the tree size grows as b^d (branching factor b , depth d), so for real time strategy games with many possible moves per turn, a full-depth Minimax search is computationally infeasible within strict move-time limits.
- Memory and time constraints: real-time play requires a decision within a fixed short window, but exhaustive Minimax may need to evaluate millions of states even at moderate depth.
- Evaluating far from terminal states: since the tree can't be searched to true game-end, the AI must cut off search at a limited depth and rely on a heuristic evaluation function, which introduces approximation error.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning adds two bounds while traversing the Minimax tree: alpha (the best value the maximizer can guarantee so far) and beta (the best value the minimizer can guarantee so far). While exploring a node, if a value is found that would make that branch worse for the player who could reach it than an already-found alternative elsewhere ($\alpha \geq \beta$), the remaining children of that node are skipped ('pruned'), because a rational opponent would

never allow the game to reach that branch anyway. This produces exactly the same decision as full Minimax while examining far fewer nodes — with good move ordering, the effective branching factor can be reduced roughly to its square root, allowing the AI to search substantially deeper within the same time budget, which is essential for meeting real-time move constraints.

iv. Designing an Evaluation Function

Since the search must be depth-limited, non-terminal states are scored by a heuristic evaluation function combining weighted game-relevant factors, for example:

- Material/unit advantage: number and strength of units or resources controlled versus the opponent's.
- Positional strength: map control, terrain advantage, proximity to strategic objectives.
- Economic factors: resource-gathering rate, production capacity, tech/upgrade level.
- Threat/safety assessment: exposure of key units or bases to attack, defensive coverage.
- Tempo/mobility: available actions, unit mobility, and initiative in the current state.

Each factor is assigned a weight reflecting its importance to winning, and the evaluation function returns a weighted sum; these weights are typically tuned through playtesting or learned via self-play so the score genuinely correlates with a player's real chance of winning from that state.

v. Strategy for Balancing Optimality and Real-Time Performance

- Use Alpha-Beta pruning with good move ordering (e.g., trying likely-best moves first, using prior search results) to maximize the depth searched within the time limit.
- Apply iterative deepening: search depth 1, then 2, then 3... so a reasonably good move is always available if time runs out mid-search, rather than risking an incomplete deep search with no usable result.
- Use a well-tuned evaluation function to keep depth-limited cutoffs meaningful, combined with quiescence search to avoid stopping evaluation in the middle of a volatile exchange (e.g., mid-battle).
- Optionally combine with transposition tables (caching previously evaluated states) to avoid recomputation when the same state is reached via different move orders. This combination lets the AI approximate optimal Minimax decisions while respecting the strict time limits required for real-time strategy gameplay, trading a small amount of true optimality for the responsiveness the game demands.